

МОСКОВСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
им. Н.Э. Баумана

Факультет “Информатика и системы управления”
Кафедра “Системы обработки информации и управления”



Дисциплина “ Парадигмы и конструкции языков программирования”

Отчет по ЛР № 4, 5

“Телеграмм бот”

Выполнил:
Студент группы ИУ5-36Б
Калмыков А.Ю.
Преподаватель:
Гапанюк Ю.Е.

Москва 2025

Листинг кода

```
import asyncio
import random

from datetime import datetime, timedelta
from telegram import Update, ReplyKeyboardMarkup
from telegram.ext import (
    Application, CommandHandler, MessageHandler,
    ContextTypes, filters, ConversationHandler
)

TOKEN = ""

GOIDA_PHOTO_URL = "https://m.gjcdn.net/game-
header/950/929009-zrkzhsjt-v4.jpg"

class FixedTimezone(timedelta):
    def __new__(cls):
        return super().__new__(cls, hours=3)

MOSCOW_TZ = datetime.now().tzinfo

def now_moscow():
    return datetime.utcnow() + FixedTimezone()

EXAMS = [
    ("ИУ6", datetime(2025, 12, 20, 9, 0)),
    ("ИУ6", datetime(2025, 12, 24, 9, 0)),
    ("ИУ5", datetime(2025, 12, 24, 14, 0)),
    ("ИУ5", datetime(2025, 12, 3, 14, 0))
]

JOKES = [
```

"Доктор после осмотра пациента: - Кости целы,
только порядок другой",

"Какой язык программирования самый дружелюбный?
Python – он змея, но не кусается!",

"Знаешь, что самое смешное при увольнении?
Находишь свою вакансию в интернете и удивляешься
тому, какие у тебя были обязанности"

]

SONGS = [

" 🎵 Rammstein – Du Hast:
<https://www.youtube.com/watch?v=W3q80d5qJio>",

" 🎵 Rammstein – Sonne:
<https://www.youtube.com/watch?v=StZcUAPRRac>",

" 🎵 Rammstein – Deutschland:
<https://www.youtube.com/watch?v=NeQM1c-XCDc>"

]

MAIN_MENU, WAITING_JOKE_CONFIRM, WAITING_SONG_CONFIRM
= range(3)

MAIN_KEYBOARD = [{"🗂️ Время до экзаменов"}, {"😏
Шутка"}], [{"🎧 Песня"}, {"💧 ГОЙДА!"}]

async def start(update: Update, context:
ContextTypes.DEFAULT_TYPE):

reply_markup = ReplyKeyboardMarkup(MAIN_KEYBOARD,
resize_keyboard=True)

await update.message.reply_text(

"Привет! Я помогу отслеживать экзамены и
немного подниму настроение 😊\n\nВыбери действие:",

reply_markup=reply_markup

)

return MAIN_MENU

```

async def timeleft_handler(update: Update, context:
ContextTypes.DEFAULT_TYPE):
    now = now_moscow()
    response_lines = []

    groups = {}
    for group, exam_dt in EXAMS:
        if group not in groups:
            groups[group] = []
        groups[group].append(exam_dt)

    for group in sorted(groups):
        lines = [f"<b>{group}</b>"]
        for exam_dt in sorted(groups[group]):
            exam_dt_local = exam_dt + FixedTimezone()
            delta = exam_dt_local - now
            if delta.total_seconds() <= 0:
                info = "☑ Экзамен уже прошёл"
            else:
                days = delta.days
                hours, remainder =
divmod(delta.seconds, 3600)
                minutes, _ = divmod(remainder, 60)
                info = f"⌚ Осталось: {days} дн.
{hours} ч. {minutes} мин."
            lines.append(f"{exam_dt.strftime('%d.%m.%
Y %H:%M')} – {info}")
        response_lines.append("\n".join(lines))

    await
update.message.reply_text("\n\n".join(response_lines)
, parse_mode="HTML")
    return MAIN_MENU

```

```

async def goida(update: Update, context:
ContextTypes.DEFAULT_TYPE):
    for _ in range(3):
        try:
            await
update.message.reply_photo(GOIDA_PHOTO_URL,
caption="ГОЙДА! 🍷")
            await asyncio.sleep(0.8)
        except Exception as e:
            await update.message.reply_text(f"Ошибка
при отправке фото: {e}")
            break
    return MAIN_MENU

# === Конечный автомат: Шутка ===
async def joke_start(update: Update, context:
ContextTypes.DEFAULT_TYPE):
    await update.message.reply_text("Точно хочешь
шутку? Она может быть очень глупой! Напиши «Да» или
«Нет».")
    return WAITING_JOKE_CONFIRM

async def joke_confirm(update: Update, context:
ContextTypes.DEFAULT_TYPE):
    text = update.message.text.strip().lower()
    if text in ["да", "yes", "ага", "конечно"]:
        joke = random.choice(JOKES)
        await update.message.reply_text(f"🤪 {joke}")
    else:
        await update.message.reply_text("Ладно,
сохраним шутку на потом 😊")
    return MAIN_MENU

# === Конечный автомат: Песня ===

```

```

async def song_start(update: Update, context:
ContextTypes.DEFAULT_TYPE):
    await update.message.reply_text("Хочешь послушать
Rammstein? Ответь «Да» или «Нет».")
    return WAITING_SONG_CONFIRM

```

```

async def song_confirm(update: Update, context:
ContextTypes.DEFAULT_TYPE):
    text = update.message.text.strip().lower()
    if text in ["да", "yes", "давай", "конечно"]:
        song = random.choice(SONGS)
        await update.message.reply_text(song)
    else:
        await update.message.reply_text("Тишина –
тоже музыка 🎵")
    return MAIN_MENU

```

```

async def unknown_input(update: Update, context:
ContextTypes.DEFAULT_TYPE):
    await update.message.reply_text("Пожалуйста,
используй кнопки или команды из меню.")
    return MAIN_MENU

```

```

def main():
    app = Application.builder().token(TOKEN).build()

    conv_handler = ConversationHandler(
        entry_points=[
            CommandHandler("start", start),
            MessageHandler(filters.Regex("^🕒 Время
до экзаменов$"), timeleft_handler),
            MessageHandler(filters.Regex("^💩
ГОЙДА!$"), goida),
            MessageHandler(filters.Regex("^🤡
Шутка$"), joke_start),

```

```

        MessageHandler(filters.Regex("^🎧
Песня$"), song_start),
    ],
    states={
        MAIN_MENU: [
            MessageHandler(filters.Regex("^📅
Время до экзаменов$"), timeleft_handler),
            MessageHandler(filters.Regex("^💧
Гойда!$"), goida),
            MessageHandler(filters.Regex("^🤔
Шутка$"), joke_start),
            MessageHandler(filters.Regex("^🎧
Песня$"), song_start),
        ],
        WAITING_JOKE_CONFIRM: [
            MessageHandler(filters.TEXT &
~filters.COMMAND, joke_confirm)
        ],
        WAITING_SONG_CONFIRM: [
            MessageHandler(filters.TEXT &
~filters.COMMAND, song_confirm)
        ],
    },
    fallbacks=[
        MessageHandler(filters.TEXT &
~filters.COMMAND, unknown_input)
    ],
    allow_reentry=True
)

app.add_handler(conv_handler)
app.run_polling()

if __name__ == "__main__":

```

```
main()
```


Скриншоты работы программы

